

Contents

CHAPTER 1: Introduction	2
1.1 Background	2
1.2 Relevance	3
1.3 Literature Survey	3
1.4 Motivation	5
1.5 Aim and Objectives	6
1.6 Technical Approach	6
CHAPTER 2: Block Schematic and Requirements	8
2.1 Introduction	8
2.2 Block Diagram	8
2.3 Requirements	10
2.4 Selection of sensors and major components	12
CHAPTER 3: System Design	13
3.1 Calculations	13
3.2 List of Components	13
CHAPTER 4: Implementation, testing and debugging	15
4.1 Implementation on bread board	15
4.2 Testing, Debugging	16
4.3 Simulation results	18
4.4 PCB design	19
4.5 final project photograph and working	23
CHAPTER 5: Results and Discussion	24
Conclusions	34
Future Scope	34
References	35

CHAPTER 1: Introduction

This chapter introduces the concept of communication protocols and their importance in embedded systems. It provides background on SPI, UART, and I2C protocols along with their real-world relevance. The chapter also discusses the motivation behind developing the Protocol Analyzer and reviews existing work in this domain. Finally, it outlines the aim, objectives, and technical approach adopted for the project.

1.1 Background

Communication protocols form the backbone of modern embedded systems, enabling microcontrollers and peripheral devices to exchange data efficiently and reliably. Among these, SPI (Serial Peripheral Interface), UART (Universal Asynchronous Receiver Transmitter), and I2C (Inter-Integrated Circuit) are widely used due to their simplicity, flexibility, and suitability for different applications. These protocols differ in terms of synchronization, speed, wiring complexity, and error handling mechanisms, making them essential topics in embedded system design and analysis.

In real-world applications, ensuring correct communication between devices is critical. Even minor issues such as signal noise, improper timing, or incorrect frame structure can lead to communication failure. Engineers often rely on debugging tools like oscilloscopes and logic analyzers to observe signals, but these tools may not provide an integrated understanding of both transmitted and received data along with protocol-level interpretation.

The Protocol Analyzer project is developed to address this gap by creating a system that not only transmits protocol frames but also analyzes how peripherals receive and respond to them. By using an ESP32-WROOM module and interfacing it with different communication ICs, the project provides a platform to study real-time communication behavior across multiple protocols.

Additionally, the integration of IoT-based monitoring allows remote visualization of communication data. By transmitting analyzed data to a web dashboard using MQTT, the system enhances accessibility and provides a more comprehensive understanding of protocol performance under different conditions.

1.2 Relevance

The study of communication protocols is fundamental in the field of Electronics and Telecommunication engineering. Almost every embedded system, from consumer electronics to industrial automation, relies on protocols such as SPI, UART, and I2C for device communication. Understanding how these protocols behave in real-world scenarios is essential for designing reliable and efficient systems.

This project is particularly relevant because it bridges the gap between theoretical learning and practical implementation. While students often study protocol characteristics in textbooks, they may not fully understand how errors occur or how devices respond to faulty transmissions. By introducing real-time analysis and visualization, this project enhances conceptual clarity and practical knowledge.

Furthermore, the integration of MQTT and web-based dashboards reflects modern industry trends, where IoT-based monitoring and remote diagnostics are increasingly important. Engineers today are expected to design systems that can not only function correctly but also provide real-time feedback and analytics.

The project also aligns with the growing demand for embedded system debugging tools. By developing a cost-effective and compact protocol analyzer, the work demonstrates how essential debugging capabilities can be implemented without relying on expensive commercial equipment.

1.3 Literature Survey

This reference was used to understand the internal architecture and peripherals of the ESP32-WROOM-32. It provided details about UART, SPI, and I2C hardware modules and their configuration. The GPIO mapping, timing behavior, and communication interfaces were studied for accurate protocol implementation. It also helped in configuring Wi-Fi for MQTT communication. Overall, it formed the base for embedded firmware development. [1] Espressif Systems – ESP32 Technical Reference Manual (2023)

This book provided background on wireless communication concepts relevant to IoT systems. It helped in understanding how data transmission reliability and noise affect communication systems. The theoretical aspects of signal degradation and error conditions were useful while designing test cases like broken and noisy frames. It also supported the concept of real-time remote monitoring. These fundamentals guided system-level design decisions. [2] Mischa Schwartz – Mobile Wireless Communications

This standard was used to understand the working of MQTT as a publish-subscribe protocol. It helped in implementing message topics, QoS levels, and client-broker communication. The project uses MQTT over WebSockets (port 8884), based on these specifications. It also guided secure communication using authentication credentials. This ensured reliable transmission of protocol data to the dashboard. [3] OASIS Standard – MQTT Version 3.1.1 (2014)

HiveMQ documentation was used to practically implement the MQTT broker setup. It helped in configuring a private broker with username and password authentication. The publish-subscribe workflow for transmitting frames from ESP32 to the dashboard was derived from this. It also guided the use of WebSocket-based MQTT for web integration. This enabled real-time visualization of received frames. [4] HiveMQ – MQTT Essentials

Arduino IDE documentation was used for writing and uploading firmware to the ESP32. It provided libraries and functions for handling SPI, UART, and I2C communication. The GPIO control for LEDs, push buttons, and display interfacing was implemented using this platform. It also supported integration with Wi-Fi and MQTT libraries. This tool simplified the overall embedded software development process. [5] Arduino IDE – Documentation

This application report helped in understanding SPI protocol structure and timing. It was used to analyze clock polarity, phase, and full-duplex communication. The working of the W25X10CL IC was studied using these concepts. It also helped in designing frame transmission cases (correct and faulty). This formed the basis for SPI waveform generation and analysis. [6] Texas Instruments – Understanding SPI Communication (2015)

This manual was used to understand I2C communication, including addressing, ACK/NACK, and bus arbitration. It guided the implementation of communication with the 24LC256 IC. The role of pull-up resistors (4.7k Ω) was derived from this specification. It also helped in interpreting received frames and error conditions. This ensured accurate I2C protocol analysis. [7] NXP Semiconductors – I2C-bus Specification (2014)

This reference provided details about asynchronous communication used in UART. It explained baud rate, start/stop bits, and parity checking. The interfacing of UART using the MAX485 IC was based on these concepts. It also helped in designing frame errors like missing bits and noise. This supported accurate UART frame analysis. [8] Microchip Technology Inc. – UART Communication Basics (2018)

Wokwi was used for simulating the ESP32-based system before hardware implementation. It helped in testing communication logic and verifying code functionality. The behavior of protocols and GPIO operations was validated virtually. This reduced debugging time during actual hardware setup. It also assisted in ensuring correct integration of components and logic. [9] Wokwi – ESP32 Simulator Documentation

1.4 Motivation

Debugging communication protocols in embedded systems is often a challenging task, especially when dealing with multiple devices and varying transmission conditions. Engineers frequently encounter issues such as incorrect data transmission, synchronization errors, and signal noise, which can be difficult to diagnose without proper tools. This challenge served as a primary motivation for developing the Protocol Analyzer.

Another motivating factor was the lack of an integrated system that simultaneously shows transmitted data, received data, and protocol interpretation. Most existing solutions either focus on signal-level analysis or high-level data monitoring, but rarely provide a unified view that connects both aspects effectively.

The idea of incorporating different transmission scenarios such as correct, broken, wrong, and noisy frames was also driven by the need to understand how communication systems behave under abnormal conditions. Studying these cases helps in designing more robust and fault-tolerant systems.

Additionally, the use of MQTT and a web-based dashboard was motivated by the need for remote monitoring and better visualization. By enabling real-time data access and historical frame tracking, the system becomes more useful for both learning and debugging purposes.

1.5 Aim and Objectives

Aim

The aim of this project is to design and implement a Protocol Analyzer using the ESP32-WROOM to analyze and visualize SPI, UART, and I2C communication at both hardware and software levels, and consider different transmission cases such as correct, wrong, broken and noise.

Objectives

- To generate protocol-specific communication frames using ESP32 and transmit them to the peripherals MAX485 (for UART), W25X10CL (for SPI), and 24LC256 (for I2C).
- To capture and analyze the behavior of SPI, UART, and I2C protocols during data transmission and reception.
- To study different transmission scenarios including correct, broken, incorrect, and noisy frames for better understanding of communication reliability.
- To compare transmitted and received data in order to detect errors and classify communication status.
- To develop an interactive web-based dashboard using HTML and CSS for visualization of waveforms, frame structures, and received data.
- To integrate MQTT protocol for enabling real-time data transfer between the ESP32 hardware and the web dashboard.
- To maintain and display historical communication data for further analysis and debugging.

1.6 Technical Approach

The development of the Protocol Analyzer involved a combination of hardware design, embedded programming, and web-based visualization. The ESP32-WROOM module was used as the central controller due to its processing capability, built-in Wi-Fi support, and compatibility with multiple communication protocols.

On the hardware side, the ESP32 was interfaced with MAX485 for UART communication, W25X10CL for SPI, and 24LC256 for I2C. Supporting components such as pull-up and pull-down resistors ensured proper signal stability, while filter capacitors reduced noise in the circuit. LEDs were used to indicate MQTT connection status and active protocol selection.

The firmware was developed using Arduino IDE, where logic was implemented to generate protocol frames, handle push-button inputs, and display transmitted waveforms on a TFT/OLED display. The system was designed to simulate different transmission conditions for analysis.

On the software side, a web dashboard was created using HTML and CSS to display received waveforms, frame structures, and response data. MQTT over WebSocket was used for communication, with a private HiveMQ broker ensuring secure data transfer. This integrated approach enabled real-time monitoring and analysis of protocol behavior.

CHAPTER 2: Block Schematic and Requirements

This chapter describes the overall architecture and working of the Protocol Analyzer system. It explains the block diagram in detail, including the role of each functional unit such as control, communication interfaces, and visualization. The chapter also outlines the hardware and software requirements necessary for system implementation and presents the criteria for selecting major components used in the design.

2.1 Introduction

The Protocol Analyzer system is designed as an integrated hardware–software platform that enables real-time analysis of SPI, UART, and I2C communication protocols. The overall architecture is centered around the ESP32-WROOM module, which acts as the main controller responsible for generating protocol frames, interfacing with peripheral devices, and transmitting analyzed data to the software dashboard. Each protocol is supported by a dedicated peripheral IC, namely MAX485 for UART, W25X10CL for SPI, and 24LC256 for I2C, allowing realistic communication scenarios to be tested and analyzed. The inclusion of push buttons enables manual selection of protocols, while supporting components such as resistors and capacitors ensure signal integrity and stable operation.

On the visualization side, the system is divided into two levels: hardware-level display and software-level dashboard. The TFT/OLED display connected to the ESP32 shows the transmitted waveform, providing immediate insight into what the controller is sending. Simultaneously, the received data from peripherals is processed and transmitted via MQTT over WebSocket to a web-based dashboard developed using HTML and CSS. This dashboard presents the received waveform, frame structure, response details, and comparison with transmitted data. The use of a private HiveMQ broker with authentication ensures secure and reliable communication.

2.2 Block Diagram of Protocol Analyzer

The block diagram represents the architecture of the Protocol Analyzer system, illustrating the interaction between control inputs, processing unit, communication interfaces, peripherals, and

visualization units. The system is designed to generate, transmit, analyze, and display communication data across multiple serial protocols in a structured manner.

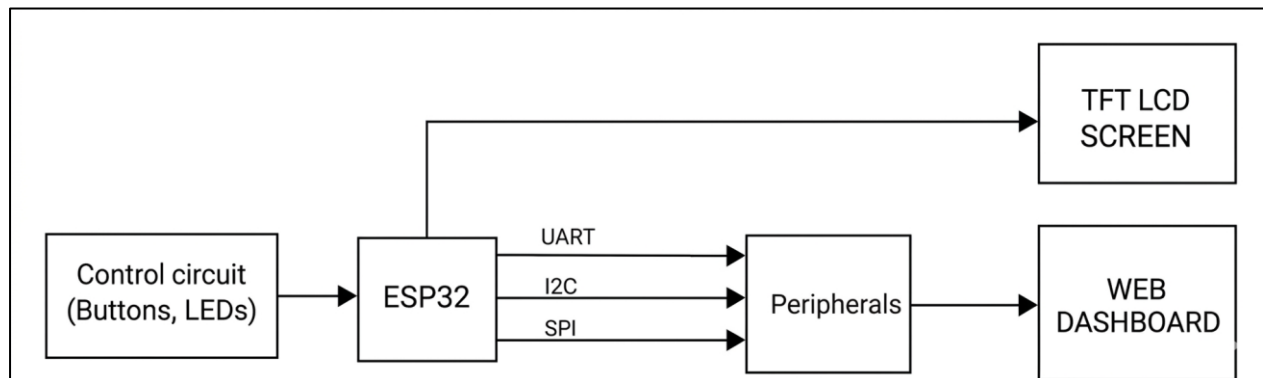


Figure 2.2: Block schematic of Protocol Analyzer

1) Control circuit block: The control circuit block provides the user interface for interacting with the system. It includes input mechanisms that allow the user to select the desired communication protocol and initiate data transmission. This block also provides visual indication of system status, such as active operation and connectivity, enabling easier monitoring and control.

2) Processing block (ESP32): The processing block acts as the central controller of the system. It is responsible for executing the core logic, including generating protocol-specific frames, handling user inputs, and managing communication across different interfaces. It also processes the received data from peripherals and prepares it for display and transmission to the software dashboard.

3) Communication interface: The communication interface includes multiple protocol selections such as UART, I2C, and SPI, which are used to transmit data from the processing unit to the connected peripherals. Each interface follows its respective communication mechanism, whether asynchronous or synchronous, allowing the system to analyze different types of serial communication.

4) Peripherals block: The peripherals block represents the external devices/ICs that receive the transmitted data and respond accordingly. These devices interpret the incoming frames based on the protocol used and generate responses that are sent back to the processing unit. This interaction enables the system to compare transmitted and received data for analysis.

5) Display block: The display unit block provides real-time visualization of the transmitted signals. It shows waveform representations corresponding to the selected protocol, allowing users

to observe how data is being sent from the processing unit. This helps in understanding signal timing and structure at the transmission level.

6) Web dashboard block: The web dashboard block serves as the software interface for advanced analysis and monitoring. It receives processed data from the system through MQTT communication mechanism and displays the received waveform, frame structure, and response details. It also enables comparison between transmitted and received data and maintains a record of past frames, making it a powerful tool for debugging and analysis.

2.3 Requirements

The following table (Table 2.3.1) lists the hardware components used in the system along with their respective purposes. These components form the physical foundation of the Protocol Analyzer and ensure proper communication, signal stability, and system functionality.

Table 2.3.1: Hardware requirements

Component / Part	Purpose
Resistors (Pull-up/Pull-down)	Used to maintain stable logic levels on communication lines and prevent floating signals.
Resistors (Current Limiting)	Used to limit current through LEDs to prevent damage and ensure safe operation.
Capacitors (High Frequency Noise Filter)	Used to filter high-frequency noise and stabilize signal lines during communication.
Capacitors (Low Frequency Noise Filter)	Used for power supply smoothing and reducing low-frequency fluctuations.
Header Pins	Used for making detachable and reliable connections between different modules and circuits.
LEDs (Red, White)	Used as visual indicators for system status such as connectivity and active protocol operation.
Push Buttons	Used to provide user input for selecting different communication protocols.
TFT LCD Display	Used to display real-time waveform of transmitted signals for visualization.
Microcontroller Module	Acts as the central processing unit for generating, transmitting, and analyzing protocol data.
Communication Interface Module (RS-485)	Used for implementing serial communication in UART mode and enabling data transmission over differential signaling.

Flash Memory Device	Used as a peripheral for SPI communication to store and retrieve data during analysis.
EEPROM Device	Used as a peripheral for I2C communication to test memory read and write operations.
Jumper Wires	Used for making flexible connections between components during prototyping.
Breadboard	Used for temporary circuit assembly and testing without soldering.
Zero PCB	Used for permanent circuit implementation after successful testing on breadboard.
Connecting Wires	Used for establishing electrical connections between components.
USB Cable	Used for powering the system and uploading code to the controller.

The following table (Table 2.3.2) outlines the software tools, technologies, and protocols used in the development of the system. These tools enable firmware development, real-time communication, and visualization through the web-based dashboard.

Table 2.3.2: Software requirements

Software / Tool	Purpose
Arduino IDE	Used for writing, compiling, and uploading firmware to the central controller for implementing protocol transmission and control logic.
Embedded C / Arduino Programming	Used to develop the logic for generating protocol frames, handling different transmission cases, and interfacing with communication protocols.
HTML	Used to design the structure of the web-based dashboard for displaying protocol data and analysis results.
CSS	Used to style and enhance the visual appearance of the dashboard, including layout and colors.
MQTT Protocol (WebSocket)	Used for real-time communication between the hardware system and the web dashboard over a network.
HiveMQ Broker (Private Server)	Used as the message broker to manage data exchange securely between publisher (device) and subscriber (dashboard).
Web Browser	Used to access and interact with the dashboard for real-time visualization and analysis of protocol data.
JSON Format	Used for structuring and transmitting data between the hardware and dashboard in a standardized format.

2.4 Selection of sensors and major components

The following table (Table 2.4) presents the selection criteria and specifications of the major components used in the system. It highlights important parameters such as voltage range, interface type, and operating conditions considered during component selection.

Table 2.4: Component selection

List Parameter	Design Requirement	Models			Units
		MAX485	W25X10CL	24LC256	
Manufacturer	--	Maxim Integrated	Microchip Technology	Winbond	--
Supply Voltage	3.3V	4.75 – 5.25	2.7 – 3.6	2.5 – 5.5	V
Interface Type	UART / SPI / I2C	UART	SPI	I2C	--
Memory Size	As required	--	1 Mbit (128 KB)	256 Kbit (32 KB)	Bits
Operating Temp	Industrial	-40 to 85	-40 to 85	-40 to 85	Celsius
ESD Protection	Robust	±15 kV	±2 kV	±4 kV	kV
Package	Compact	DIP	SOIC-8	DIP	--
Type	Communication / Memory	RS-485 Transceiver	Flash	EEPROM	--
Operating Frequency / Data Rate	Efficient	Up to 2.5 Mbps	104 MHz	400 kHz (standard), up to 1 MHz (fast mode)	Hz / bps

CHAPTER 3: System Design

This chapter focuses on the detailed design aspects of the system, including necessary calculations and component selection. It explains the design considerations involved in ensuring proper operation of the hardware components. Additionally, it provides a complete list of components used in the project along with their specifications, forming the basis for system implementation.

3.1 Calculations

Supply voltage (V) = 3.3 V

Resistor (R) = 330 Ω

LED forward voltages:

Red LED: $V_f = 2.0$ V

White LED: $V_f = 2.4$ V

Formula used (Ohm's Law):

$$I = \frac{(V - V_f)}{R} \quad \dots(1.1)$$

1) Red LED Calculation

$$I = \frac{3.3 - 2.0}{330}$$

$$I = 0.00394 \text{ A}$$

$$I = 3.94 \text{ mA}$$

2) Similarly, for white LED

$$I = 0.00273 \text{ A}$$

$$I = 2.73 \text{ mA}$$

3.2 List of Components

The following table (Table 3.2) provides a detailed list of all components used in the project along with their specifications and model numbers. This list serves as a reference for system assembly and implementation.

Table 3.2: List of components required in project.

Sr. No	Name and description of part	Value / model number
1	Resistors (Pullup/Pulldown)	4.7k Ω
2	Resistors (Current Limiting)	330 Ω
3	Capacitors (High Freq Noise Filter)	100nF
4	Capacitors (Low Freq Noise Filter)	10uF
5	Header Pins	Male Connectors
6	LEDs (Red, White)	2V, 2.4V
7	Push Buttons	1mm x 1mm Tactile Switches
8	2.8" TFT LCD	ILI9341
9	ESP32	Wroom Dev Module (38 pins)
10	RS 485 Transceiver	MAX485
11	Flash	W25X10CL
12	EEPROM	24LC256
13	Jumper Wires	M2F, M2M Jumper Wires
14	Breadboard	8cm x 5cm
15	Zero PCB	8cm x 4.5cm
16	Wires	Single Core Wires
17	USB Cable	Type A to Type C

CHAPTER 4: Implementation, testing and debugging

This chapter presents the practical implementation of the Protocol Analyzer system along with the testing and debugging procedures followed. It explains how the system was initially implemented on a breadboard and later verified through simulations. The chapter also discusses the behavior of different communication protocols under various transmission scenarios and highlights the PCB design and final system integration.

4.1 Implementation on bread board

The following image (Fig. 4.1) represents the successful implementation of the Protocol Analyzer system on a breadboard setup. The ESP32-WROOM module was interfaced with peripherals such as MAX485 (UART), W25X10CL (SPI), and 24LC256 (I2C) to analyze real-time communication across different protocols. Proper pull-up/pull-down resistors and filtering components were used to ensure stable signal transmission. Each connection and communication line was carefully tested and verified before proceeding to the final PCB implementation, ensuring accuracy and reliability of the system.

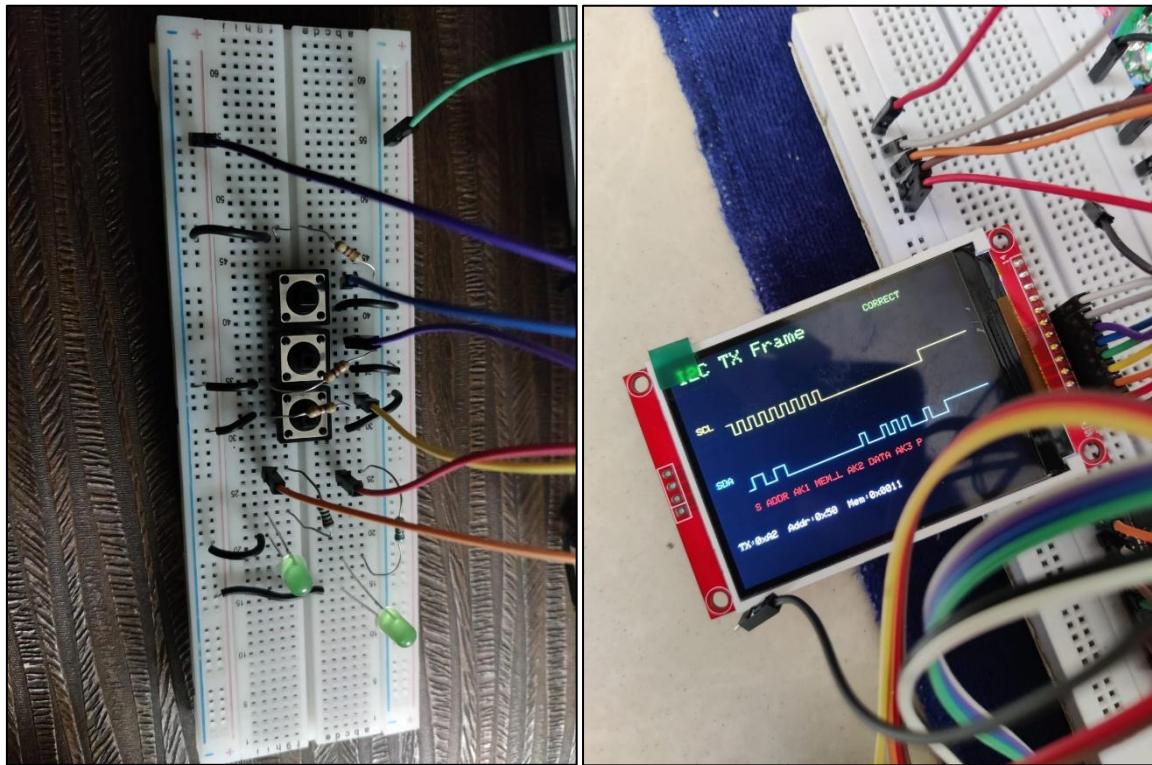


Figure 4.1: Breadboard testing

4.2 Testing, Debugging

Serial monitor output (Considering each protocol):

```

--- UART RS485 ---
UART TX: 0xA1 parity:1 [CORRECT]
UART RX: 0xA1 ACK [CORRECT]
TX → {"protocol":"UART","frame":{"start":0,"data":"0xA1","parity":1,"stop":1}}
RX → {"protocol":"UART","frame":{"start":0,"data":"0xA1","parity":1,"stop":1},"response":{"ack":"ACK","status":"PASS","mode":"CORRECT"}}

UART TX: 0xA2 parity:1 [CORRECT]
UART RX: 0xA2 ACK [CORRECT]
TX → {"protocol":"UART","frame":{"start":0,"data":"0xA2","parity":1,"stop":1}}
RX → {"protocol":"UART","frame":{"start":0,"data":"0xA2","parity":1,"stop":1},"response":{"ack":"ACK","status":"PASS","mode":"CORRECT"}}

UART TX: 0xA3 parity:0 [NOISE ]
UART RX: 0xAB NACK [NOISE ]
TX → {"protocol":"UART","frame":{"start":0,"data":"0xA3","parity":0,"stop":1}}
RX → {"protocol":"UART","frame":{"start":0,"data":"0xAB","parity":1,"stop":1},"response":{"ack":"NACK","status":"FAIL","mode":"NOISE  "}}

UART TX: 0xA4 parity:1 [NOISE ]
UART RX: 0xAC NACK [NOISE ]
TX → {"protocol":"UART","frame":{"start":0,"data":"0xA4","parity":1,"stop":1}}
RX → {"protocol":"UART","frame":{"start":0,"data":"0xAC","parity":0,"stop":1},"response":{"ack":"NACK","status":"FAIL","mode":"NOISE  "}}

UART TX: 0xA5 parity:0 [BROKEN ]
UART RX: 0x00 NACK [BROKEN ]
TX → {"protocol":"UART","frame":{"start":0,"data":"0xA5","parity":0,"stop":1}}
RX → {"protocol":"UART","frame":{"start":0,"data":"0x00","parity":0,"stop":1},"response":{"ack":"NACK","status":"FAIL","mode":"BROKEN  "}}

```

Figure 4.2.1: UART protocol serial monitor output

```

--- I2C EEPROM ---
I2C WRITE: addr:0x0010 data:0xA1 [CORRECT]
I2C READ:  addr:0x0010 data:0xA1 PASS [CORRECT]
TX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"W","mem_addr":"0x0010","ack2":"ACK","data":"0xA1","ack3":"ACK","stop":1}}
RX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"R","mem_addr":"0x0010","ack2":"ACK","data":"0xA1","ack3":"ACK","stop":1}}

I2C WRITE: addr:0x0011 data:0xA2 [CORRECT]
I2C READ:  addr:0x0011 data:0xA2 PASS [CORRECT]
TX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"W","mem_addr":"0x0011","ack2":"ACK","data":"0xA2","ack3":"ACK","stop":1}}
RX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"R","mem_addr":"0x0011","ack2":"ACK","data":"0xA2","ack3":"ACK","stop":1}}

I2C WRITE: addr:0x0012 data:0xA3 [NOISE ]
I2C READ:  addr:0x0012 data:0x83 FAIL [NOISE ]
TX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"W","mem_addr":"0x0012","ack2":"ACK","data":"0xA3","ack3":"NACK","stop":1}}
RX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"R","mem_addr":"0x0012","ack2":"ACK","data":"0x83","ack3":"NACK","stop":1}}

I2C WRITE: addr:0x0013 data:0xA4 [NOISE ]
I2C READ:  addr:0x0013 data:0xE4 FAIL [NOISE ]
TX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"W","mem_addr":"0x0013","ack2":"ACK","data":"0xA4","ack3":"NACK","stop":1}}
RX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"R","mem_addr":"0x0013","ack2":"ACK","data":"0xE4","ack3":"NACK","stop":1}}

I2C WRITE: addr:0x0014 data:0xA5 [BROKEN ]
I2C READ:  addr:0x0014 data:0x00 FAIL [BROKEN ]
TX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"W","mem_addr":"0x0014","ack2":"ACK","data":"0xA5","ack3":"NACK","stop":1}}
RX → {"protocol":"I2C","frame":{"start":1,"slave_addr":"0x50","ack1":"ACK","rw":"R","mem_addr":"0x0014","ack2":"ACK","data":"0x00","ack3":"NACK","stop":1}}

```

Figure 4.2.2: I2C protocol serial monitor output


```

Output  Serial Monitor X
Message (Enter to send message to 'ESP32-WROOM-DA Module' on 'COM3')

--- SPI Flash ---
SPI WRITE: addr:0x000010 data:0xA1 [CORRECT]
SPI READ: addr:0x000010 data:0xA1 PASS [CORRECT]
TX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000010","data":"0xA1","cs_end":"HIGH"}}
RX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000010","data":"0xA1","cs_end":"HIGH"}}

SPI WRITE: addr:0x000011 data:0xA2 [CORRECT]
SPI READ: addr:0x000011 data:0xA2 PASS [CORRECT]
TX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000011","data":"0xA2","cs_end":"HIGH"}}
RX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000011","data":"0xA2","cs_end":"HIGH"}}

SPI WRITE: addr:0x000012 data:0xA3 [NOISE ]
SPI READ: addr:0x000012 data:0xE3 FAIL [NOISE ]
TX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000012","data":"0xA3","cs_end":"HIGH"}}
RX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000012","data":"0xE3","cs_end":"HIGH"}}

SPI WRITE: addr:0x000013 data:0xA4 [NOISE ]
SPI READ: addr:0x000013 data:0xAC FAIL [NOISE ]
TX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000013","data":"0xA4","cs_end":"HIGH"}}
RX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000013","data":"0xAC","cs_end":"HIGH"}}

SPI WRITE: addr:0x000014 data:0xA5 [BROKEN ]
SPI READ: addr:0x000014 data:0x00 FAIL [BROKEN ]
TX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000014","data":"0xA5","cs_end":"HIGH"}}
RX - {"protocol":"SPI","frame":{"cs":"LOW","write_enable":"0x06","cmd":"0x02","addr":"0x000014","data":"0x00","cs_end":"HIGH"}}

```

Figure 4.2.3: SPI protocol serial monitor output

During debugging, all three communication protocols - UART, SPI, and I2C - were tested under different transmission scenarios including correct, noisy, broken, and wrong cases to evaluate system behavior. It was observed that in the correct case, the transmitted and received data matched accurately, resulting in successful acknowledgment and PASS status. In noisy conditions, data corruption occurred due to disturbances, leading to mismatched values and failure responses. In broken cases, incomplete or improperly structured frames resulted in invalid or zero data at the receiver, indicating failure in communication. In wrong cases, although the frame structure was valid, incorrect data transmission caused mismatches and NACK responses. These observations helped in identifying issues related to timing, data integrity, and synchronization across all protocols. As a corrective measure, whenever, cases such as these, i.e. noisy, broken, or wrong, the frame must be retransmitted.

4.3 Simulation results

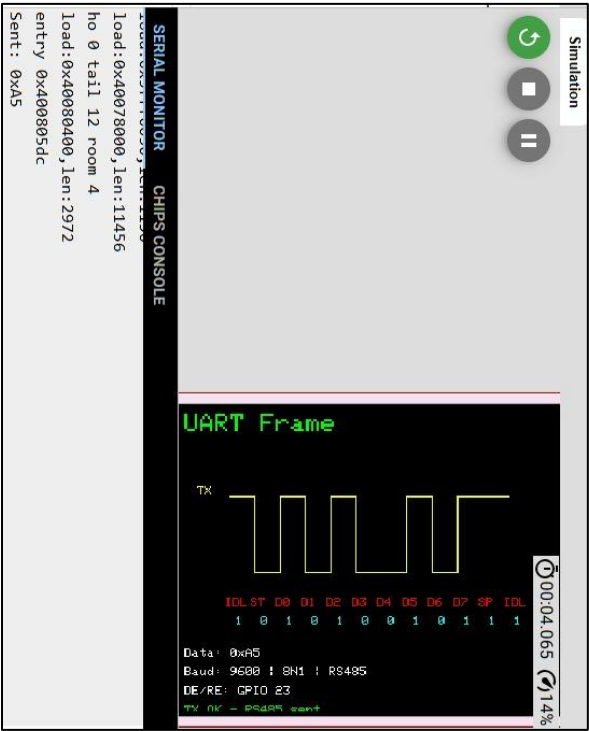


Figure 4.3.1: UART protocol simulation

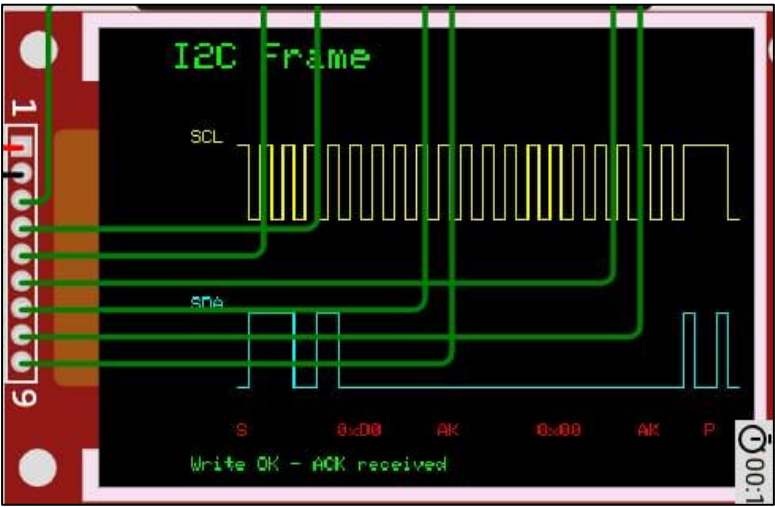


Figure 4.3.2: I2C protocol simulation

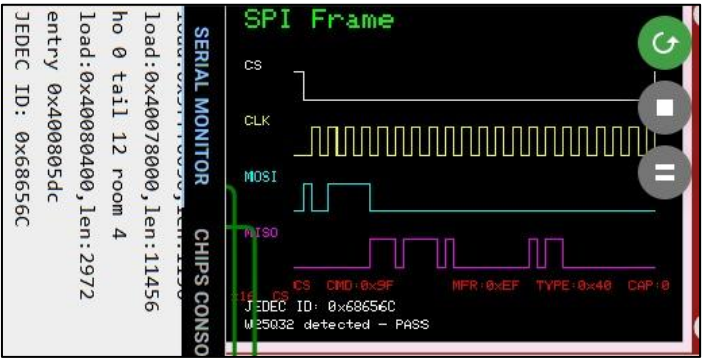


Figure 4.3.3: SPI protocol simulation

The UART protocol simulation was carried out using the Wokwi platform to analyze asynchronous serial communication behavior. (Refer fig. 4.3.1) In this simulation, data transmission was performed without a clock signal, relying on predefined baud rate timing. The waveform clearly shows the presence of start bit, data bits, and stop bit, which are essential components of UART communication. The simulation helped in verifying timing accuracy, bit alignment, and overall frame structure before implementing the protocol on hardware.

The I2C protocol was simulated to study synchronous communication using shared clock and data lines. (Refer fig. 4.3.2) The waveform illustrates the SCL (clock) and SDA (data) signals, along with start and stop conditions, addressing, and acknowledgment bits. The simulation demonstrates how data is transmitted in a structured format and how acknowledgment from the receiving device confirms successful communication. This simulation was useful in understanding multi-device communication and ensuring correct frame sequencing before hardware implementation.

The SPI protocol simulation was performed to analyze high-speed synchronous communication between master and slave devices. (Refer fig. 4.3.3) The waveform shows the key signals involved, including chip select, clock and data transmission signals. The simulation highlights how data is shifted in synchronization with the clock and how full-duplex communication is achieved. This helped in validating the timing relationships and frame structure of SPI communication, ensuring accurate implementation in the actual system.

4.4 PCB design

The system was tested by considering each communication protocol individually to ensure accurate analysis and proper functioning. SPI, UART, and I2C protocols were tested independently by transmitting different types of frames such as correct, wrong, broken, and noisy cases (Refer Fig. 4.4.1). For each protocol, the transmitted waveform on the hardware display and the received data on the web dashboard were carefully observed and verified. This approach ensured that protocol-specific issues could be identified and resolved without interference from other communication interfaces.

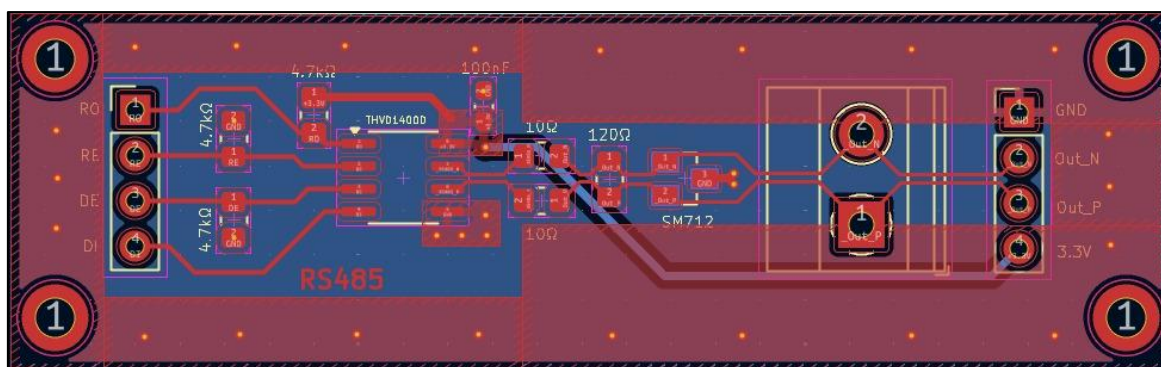


Figure 4.4.1(a): UART protocol testing

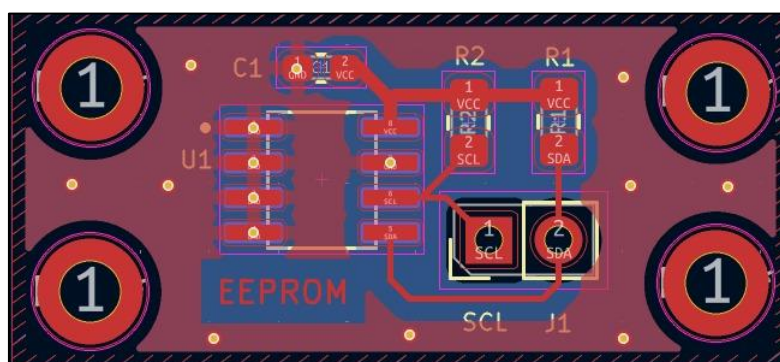


Figure 4.4.1(b): I2C protocol testing

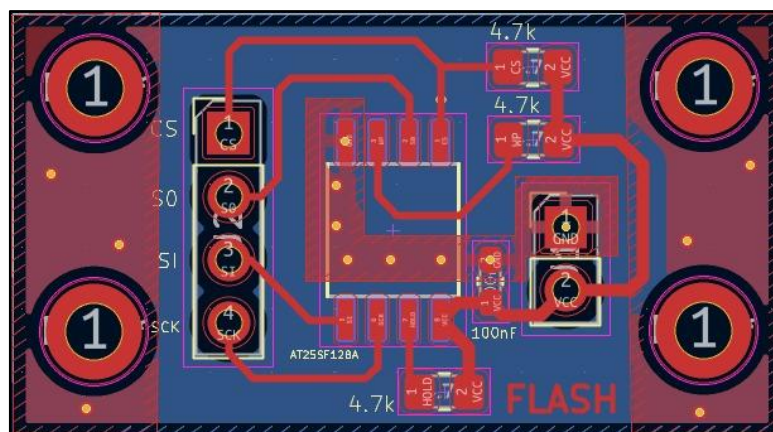


Figure 4.4.1(c): SPI protocol testing

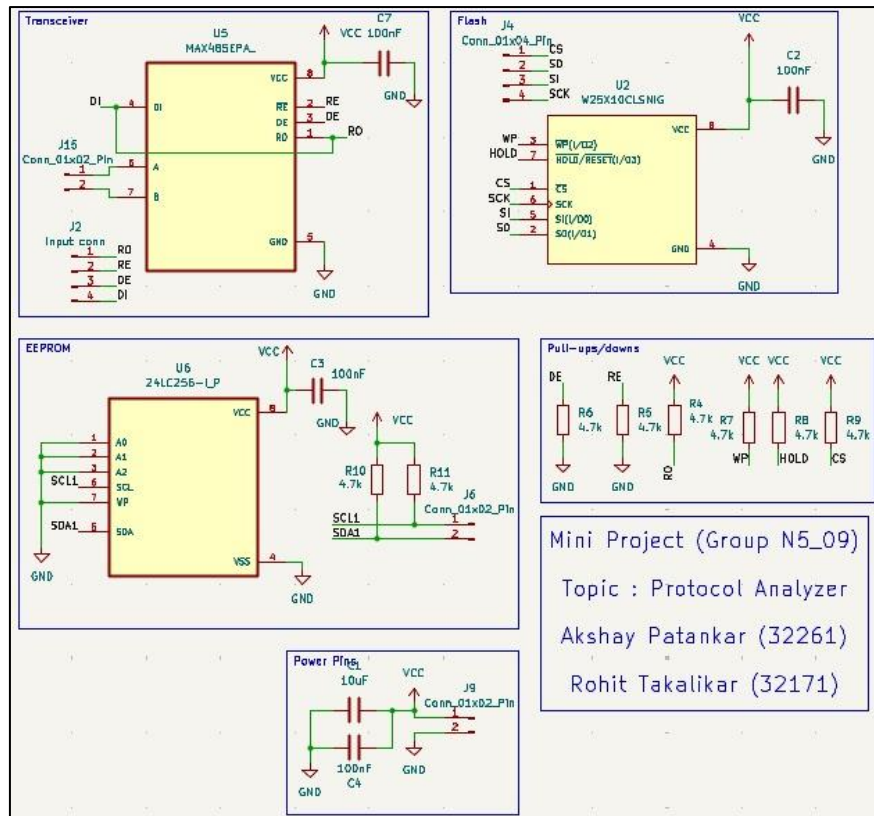


Figure 4.4.2: Final Schematic

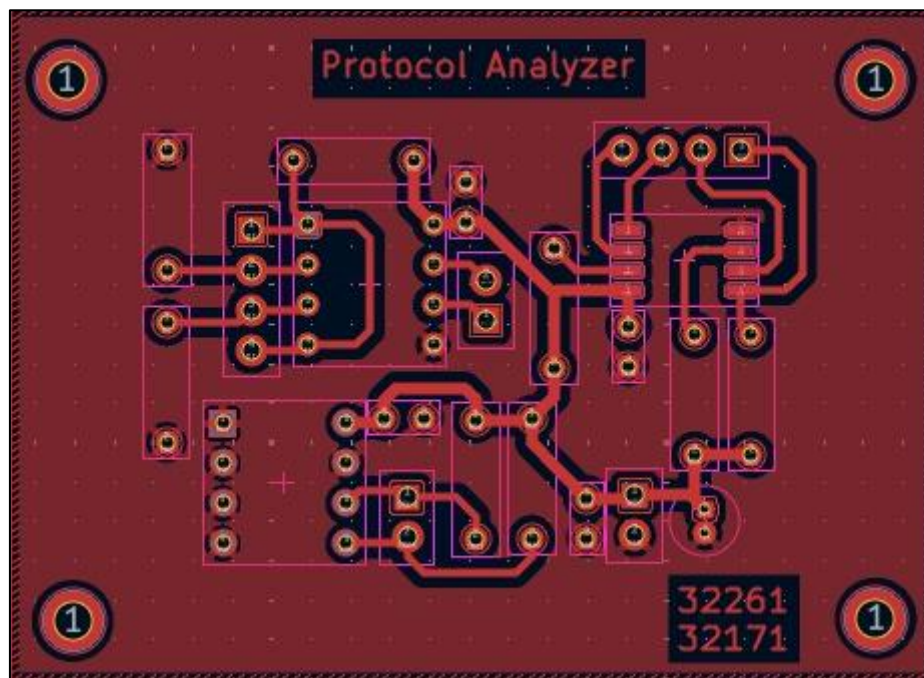


Figure 4.4.3: Final PCB Layout

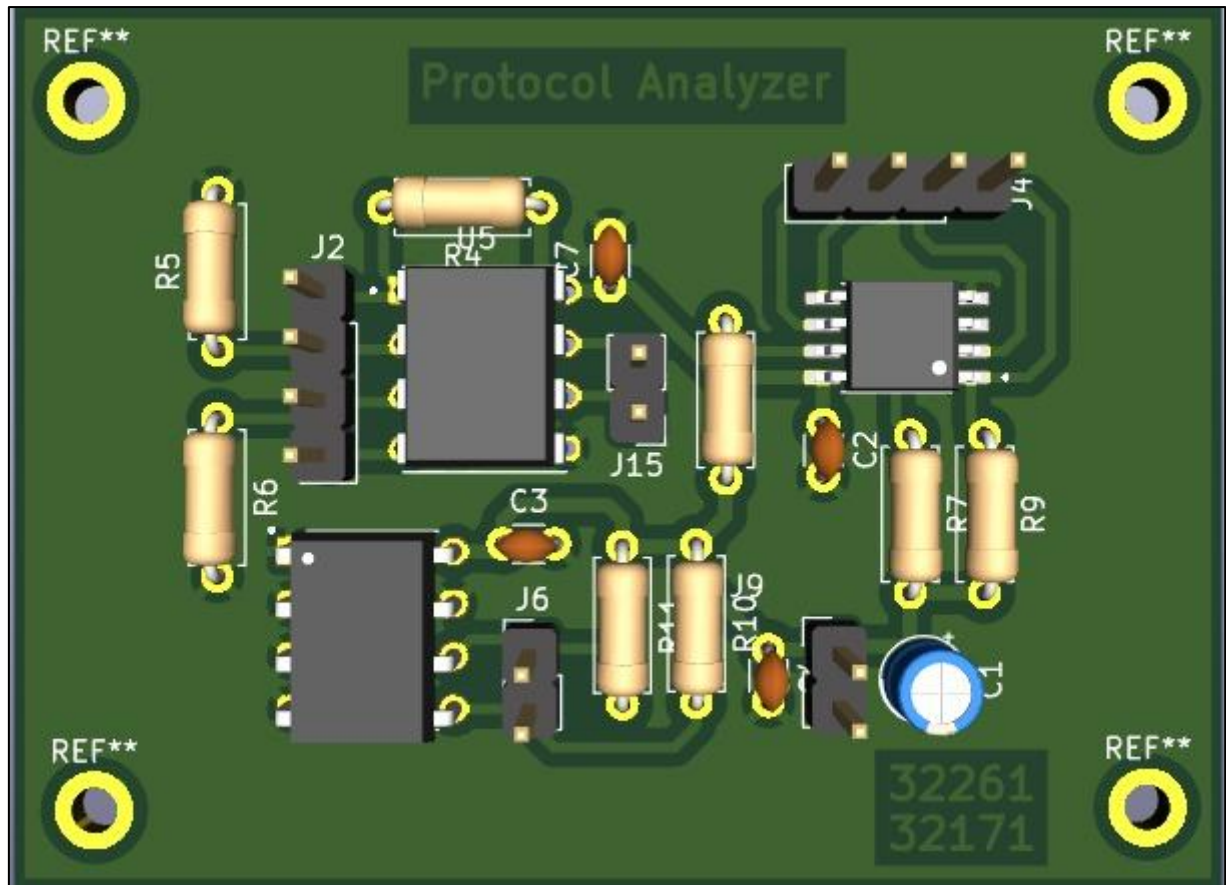


Figure 4.4.4: Final PCB 3D – View

The schematic, PCB layout, and 3D view together represent the complete hardware design of the Protocol Analyzer system. The schematic (Refer fig. 4.4.2) illustrates the connections between the ESP32, peripheral devices, and supporting components, ensuring correct signal flow and interfacing. Based on this schematic, the PCB layout (Refer fig. 4.4.3) was designed to achieve a compact, organized, and noise-resistant physical implementation of the circuit. The 3D view (Refer fig. 4.4.4) provides a realistic representation of the assembled board, helping visualize component placement and overall structure. Together, these stages ensure a reliable transition from conceptual design to practical hardware realization.

4.5 final project photograph and working

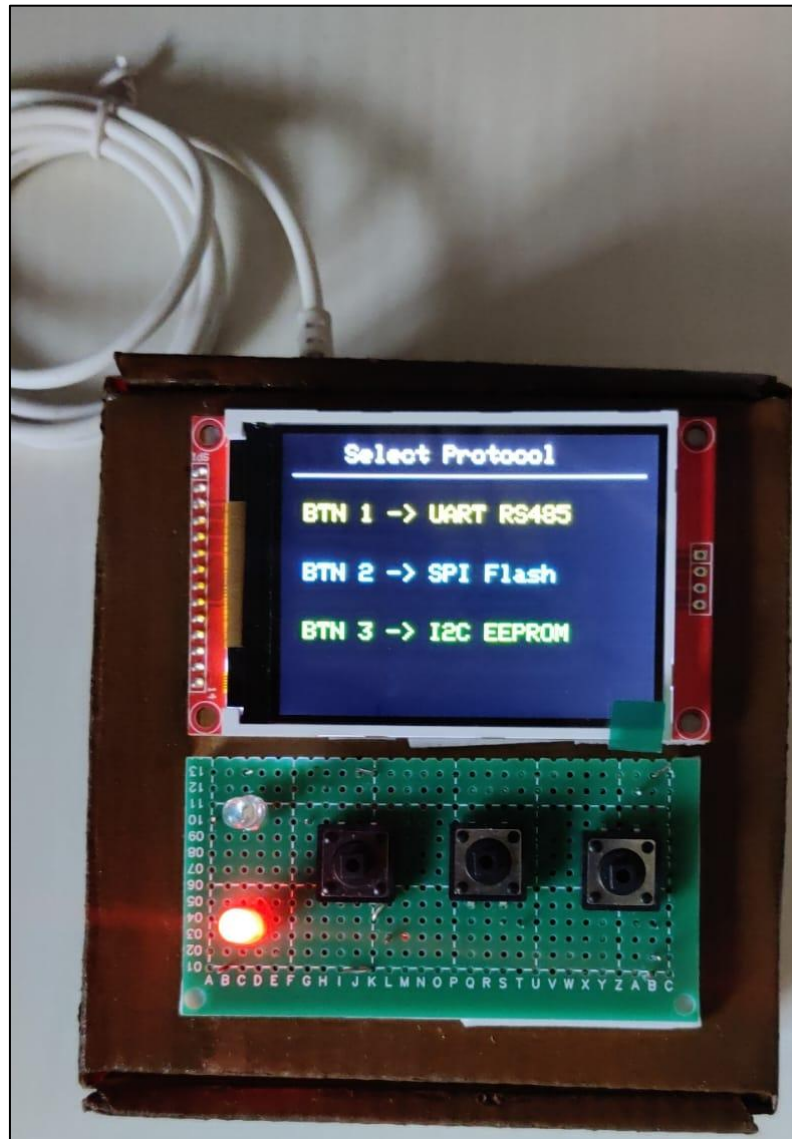


Figure 4.5: Project working

User interaction with the system is achieved through push-button inputs, which allow selection of the desired communication protocol for analysis. Visual feedback is provided using LED indicators, where one LED (red) represents successful establishment of MQTT communication and another (white) indicates whether any protocol is currently active (Refer fig. 4.5). The received data from the selected protocol is transmitted to a web-based dashboard using MQTT, where it is displayed in real time along with frame structure and response details, enabling efficient monitoring and analysis.

CHAPTER 5: Results and Discussion

This chapter analyzes the performance of the developed Protocol Analyzer system under different communication conditions. It presents the observed results for SPI, UART, and I2C protocols, including waveform visualization, frame structure, and response analysis. The chapter also discusses the system's ability to detect errors and compares transmitted and received data, providing insights into protocol behavior and reliability.

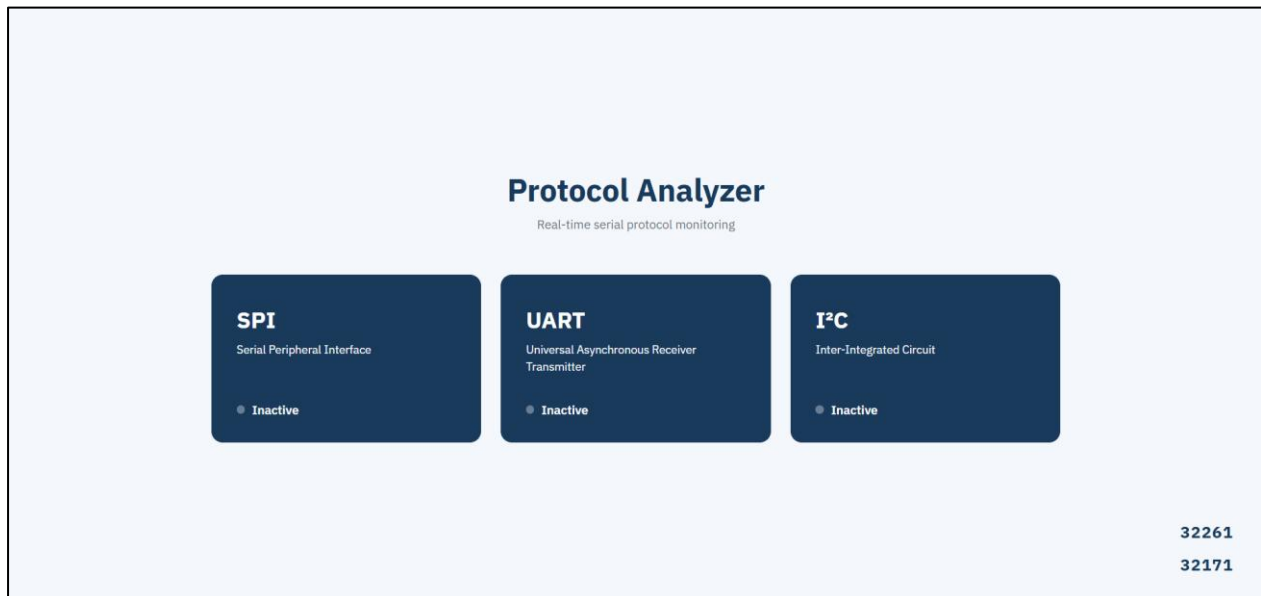


Figure 5.1: Protocol Analyzer dashboard

The developed Protocol Analyzer system was tested for SPI, UART, and I2C communication by transmitting different types of frames and observing the corresponding responses from the connected peripherals (Refer fig. 5.1). The results were visualized on the web dashboard, where separate sections were designed for each protocol. For every protocol, the dashboard displayed the received waveform, frame structure, transmitted vs received comparison, and response information.

In order to evaluate the reliability and behavior of communication protocols under varying conditions, the system was designed to simulate and analyze four different transmission scenarios: correct, wrong, broken, and noisy frames. These cases were implemented at the ESP32 transmission level, and the corresponding responses from the peripheral device were displayed on the dashboard. Used UART protocol for demonstration, where the transmitted waveform, received data, and response status were analyzed together.

5.1 UART protocol



Figure 5.2.1: UART transmitted frame for CORRECT case

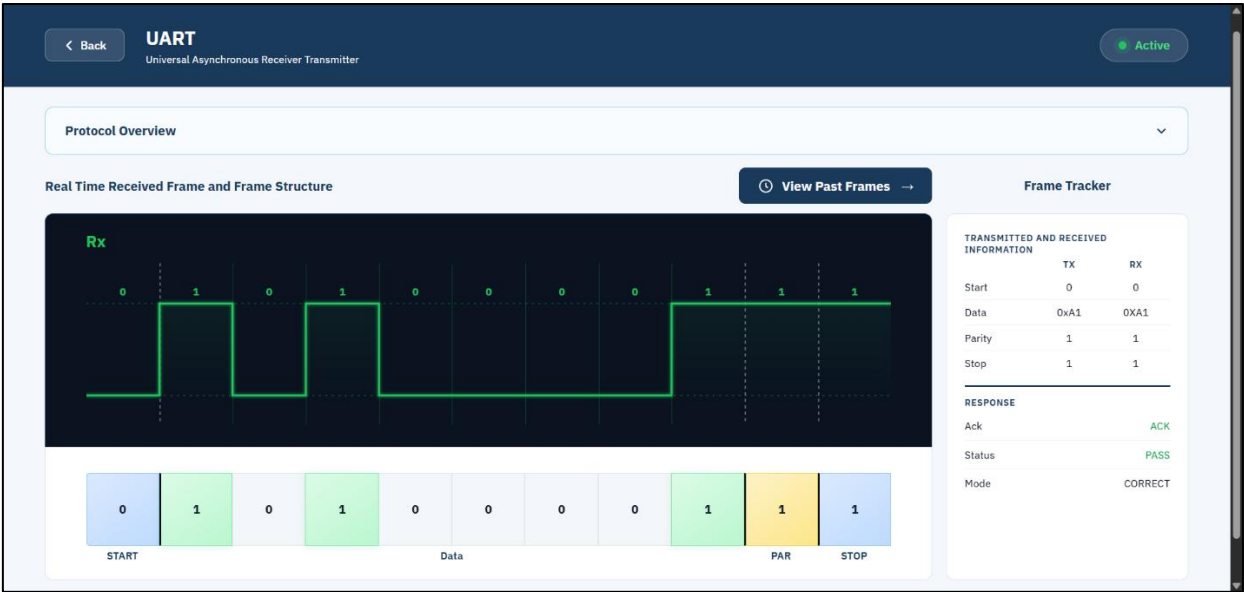


Figure 5.2.2: UART received frame for CORRECT case

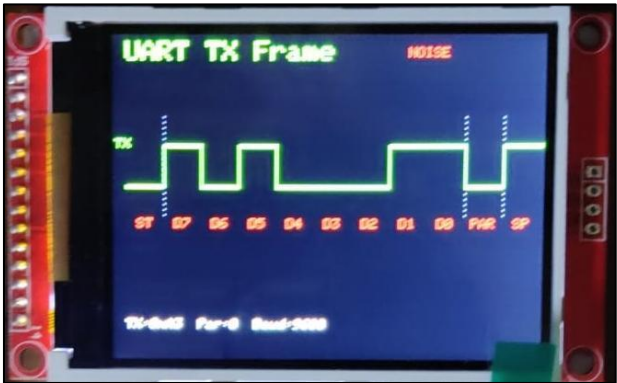


Figure 5.3.1: UART transmitted frame for NOISE case

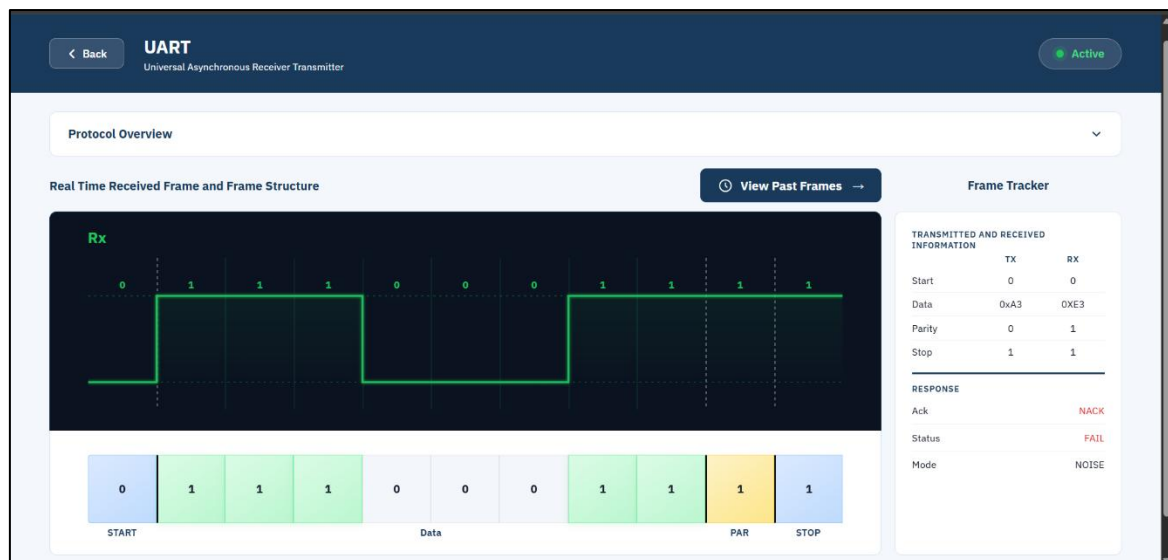


Figure 5.3.2: UART received frame for NOISE case

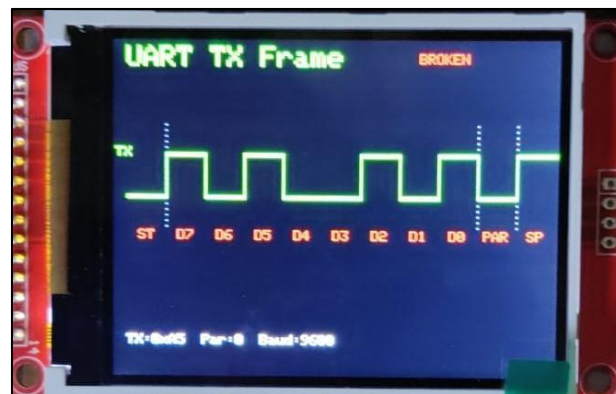


Figure 5.4.1: UART transmitted frame for BROKEN case

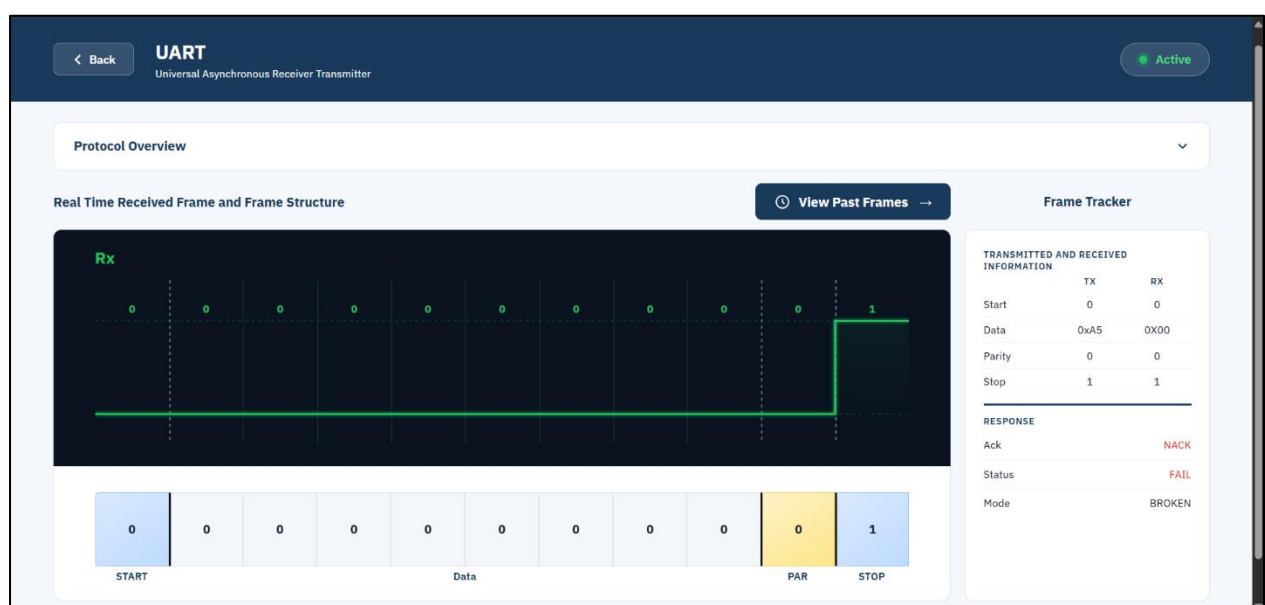


Figure 5.4.2: UART received frame for BROKEN case



Figure 5.5.1: UART transmitted frame for WRONG case

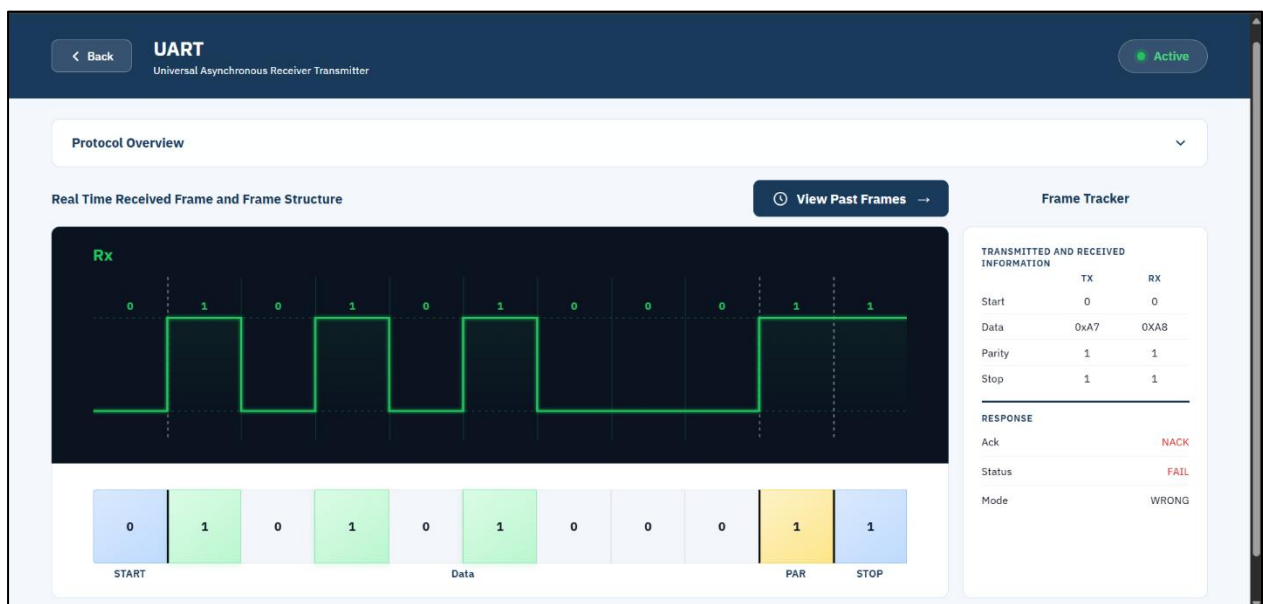


Figure 5.5.2: UART received frame for WRONG case

In the **correct** transmission case (figure 5.2), the data frame generated by the ESP32 strictly follows the UART protocol format, including proper start bit, data bits, parity bit and stop bit. The transmitted data matches exactly with what the peripheral receives, resulting in successful communication. On the dashboard, this is reflected by identical transmitted and received values, a valid acknowledgment, and a PASS status. This case serves as the reference scenario, confirming that the system operates correctly under ideal conditions.

In the **noisy** transmission case (figure 5.3), external disturbances or simulated noise are introduced into the signal, affecting the integrity of the transmitted data. Noise can cause bit flips or timing inconsistencies, leading to discrepancies between transmitted and received values. Even if the frame structure remains intact, the data may be corrupted due to interference. The dashboard reflects this condition through mismatched data, unstable waveform representation, and error

responses such as NACK or FAIL. This case demonstrates the impact of real-world signal disturbances on communication reliability.

In the **broken** transmission case (figure 5.4), the frame structure itself is disrupted. This may include missing bits, incomplete frames, or incorrect placement of start or stop bits. Such errors prevent the peripheral from properly interpreting the data frame. Consequently, the received output may be incomplete, corrupted, or not recognized at all. On the dashboard, this is observed as irregular or truncated waveform patterns and incorrect frame decoding, along with a failure status. This case highlights the importance of proper framing in serial communication.

In the **wrong** transmission case (figure 5.5), the ESP32 intentionally sends incorrect or mismatched data, such as altered data bits or invalid frame content. Although the frame structure may still be syntactically correct, the actual data differs from what the peripheral expects. As a result, the received data does not match the transmitted data, leading to an error condition. On the dashboard, this mismatch is clearly visible in the comparison section, and the response typically indicates a NACK or FAIL status, demonstrating how logical data errors are detected.

Overall, these four cases provide a comprehensive understanding of how communication protocols behave under both ideal and faulty conditions. By visualizing these scenarios on the dashboard and correlating them with the UART waveform and response, the system effectively illustrates error detection and protocol robustness in embedded communication systems.

5.2 I2C protocol

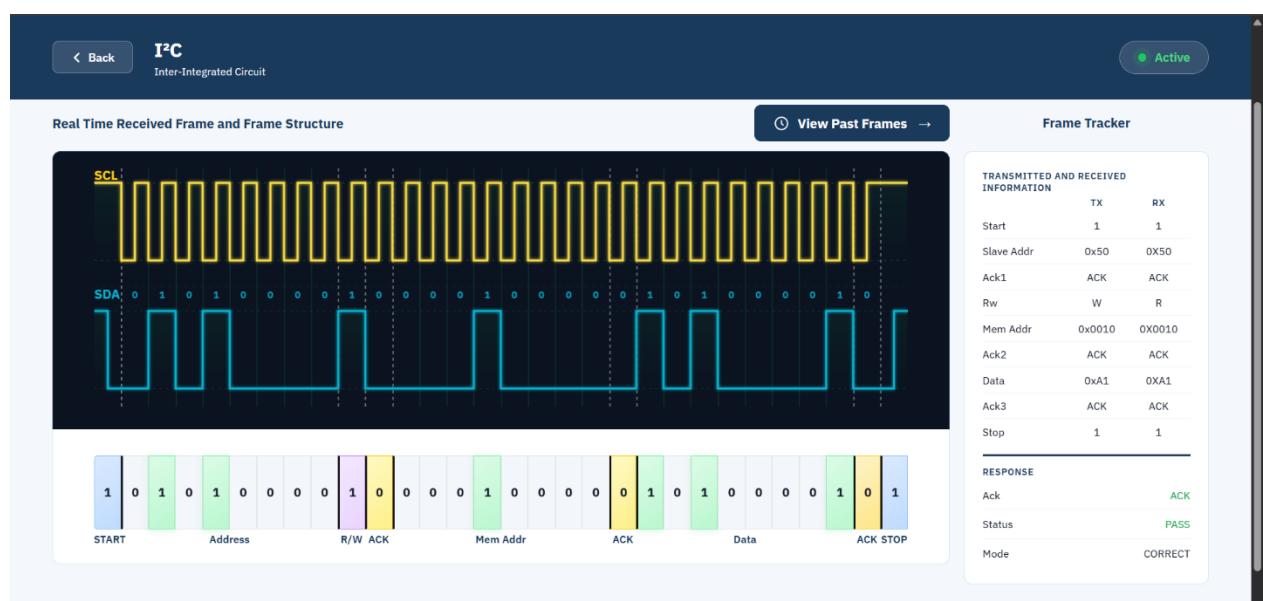


Figure 5.6: I2C frame for CORRECT case



Figure 5.7: I2C frame for NOISE case



Figure 5.8: I2C frame for BROKEN case



Figure 5.9: I2C frame for WRONG case

For the I2C protocol, the analyzer effectively captured the behavior of SDA and SCL lines and displayed the corresponding frame structure, including start condition, address, r/w, memory address, data bits, and acknowledgment bits. The dashboard visualization helped in understanding how acknowledgment signals from the slave device indicate successful communication. In cases where errors were intentionally introduced, the absence of acknowledgment or incorrect data was clearly reflected in the response section, validating the system's ability to detect faults.

5.3 SPI protocol



Figure 5.10: SPI frame for CORRECT case

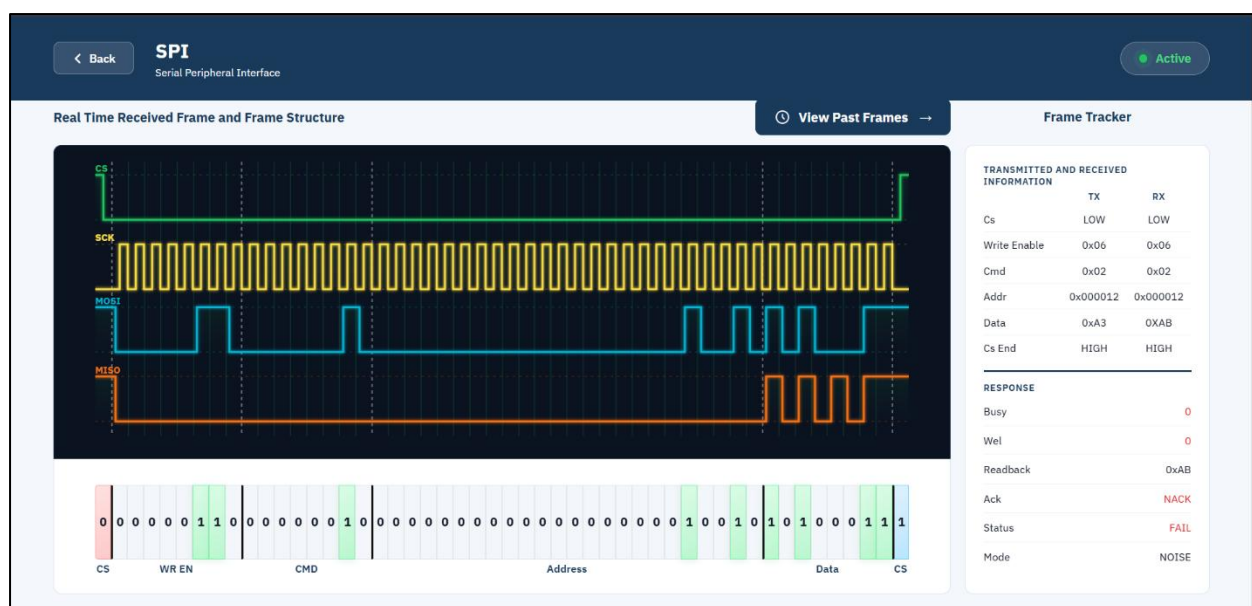


Figure 5.11: SPI frame for NOISE case

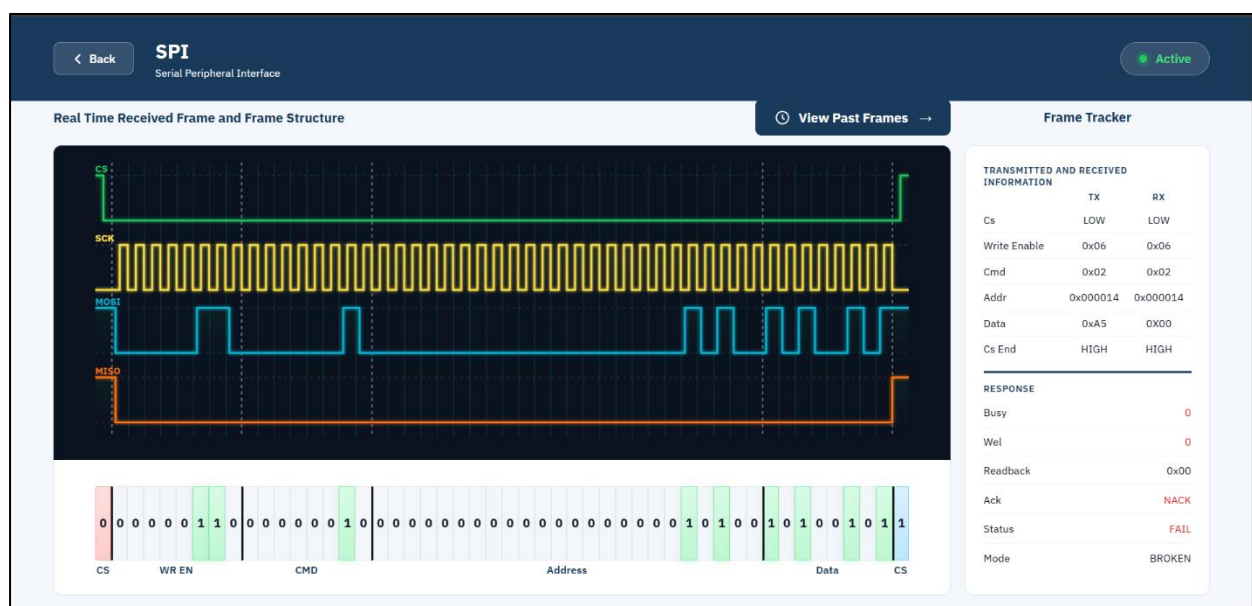


Figure 5.12: SPI frame for BROKEN case

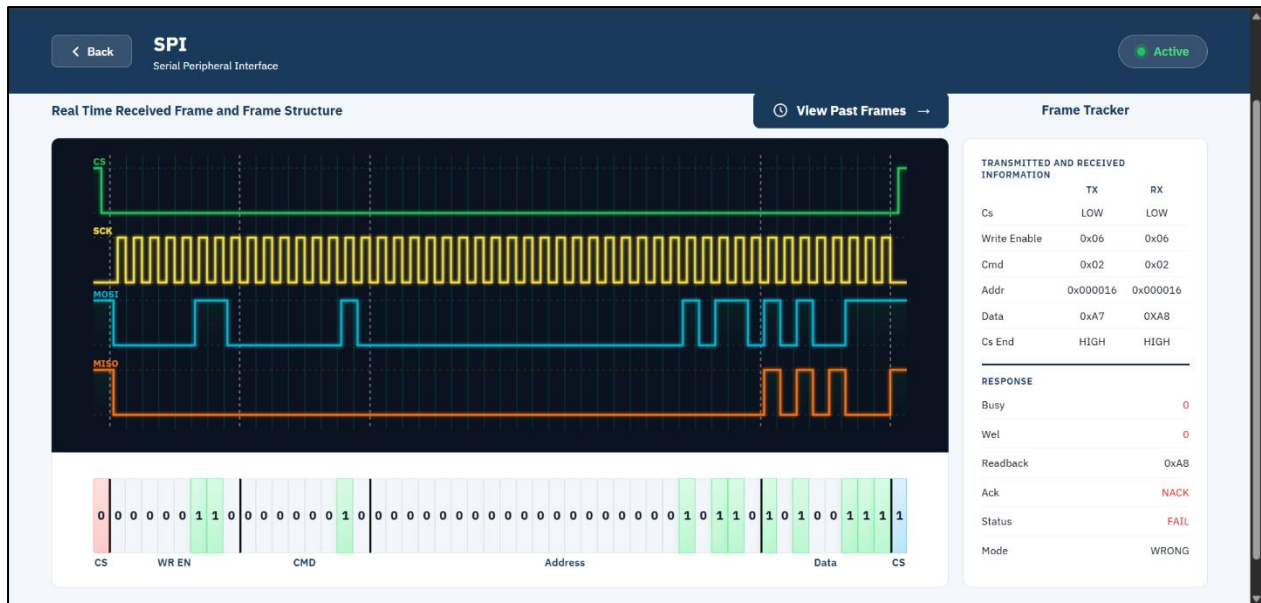


Figure 5.13: SPI frame for WRONG case

For the SPI protocol, the waveform representation clearly illustrates the CS, SCK, MOSI, and MISO signals. The frame structure displayed below the waveform provides a bit-level breakdown of the transmitted data, including write enable, command, address, and data fields. The response window shows parameters such as busy status, acknowledgment, readback data, and overall status. In correct transmission cases, the transmitted and received values matched perfectly, resulting in an ACK response and PASS status. In contrast, faulty transmissions such as noisy or incorrect frames resulted in mismatches, which were reflected as NACK and FAIL status in the response section.

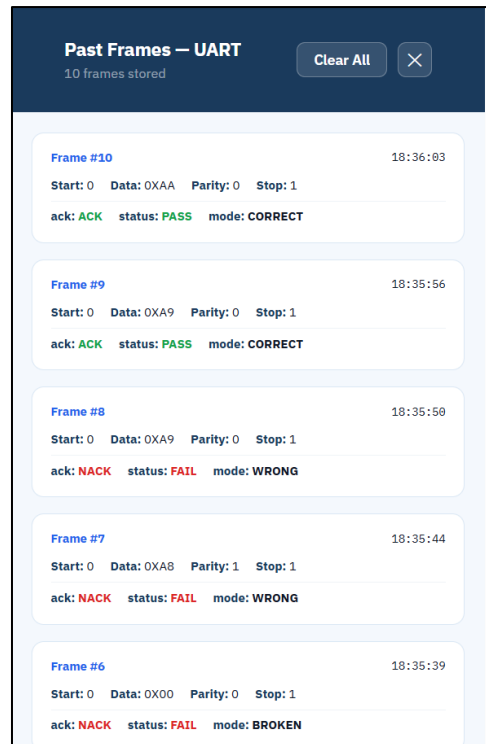


Figure 5.14: Past frames storage

An important feature of the system is the “Past Frames” section, which stores previously received frames along with their timestamps, data values, acknowledgment status, and overall result. (figure 5.14) This feature proved useful for tracking communication history and comparing multiple transmission attempts. It allows users to analyze patterns in errors and verify improvements after modifying transmission conditions.

Overall, the results demonstrate that the Protocol Analyzer successfully provides a comprehensive view of communication behavior by combining waveform visualization, frame decoding, response tracking, and historical logging. The comparison between transmitted and received data is particularly effective in identifying errors and understanding protocol behavior in both normal and abnormal conditions.

Conclusion

The Protocol Analyzer system was successfully designed and implemented to analyze SPI, UART, and I2C communication using the ESP32-WROOM module. The system effectively demonstrated real-time transmission and reception of protocol frames, along with visualization on both the display and web dashboard.

The use of MQTT over WebSocket for communication between hardware and dashboard ensured reliable and efficient data transfer. The dashboard interface provided clear visualization of waveforms, frame structures, and response information, making the analysis process intuitive and user-friendly. Features such as transmitted vs received comparison and past-frame storage further enhanced the usability of the system.

The project successfully met its objectives by providing a compact, cost-effective, and efficient protocol analysis tool. It also strengthened understanding of embedded communication protocols and demonstrated the practical application of IoT-based monitoring systems.

Future Scope

The Protocol Analyzer can be further enhanced by incorporating additional communication protocols such as CAN, LIN, or advanced SPI modes to broaden its applicability. Adding automatic protocol detection and intelligent error classification using machine learning techniques can significantly improve the system's analytical capabilities.

The dashboard can be upgraded with advanced visualization features such as timing analysis, and graphical statistics to provide deeper insights into communication behaviour. Data export options and cloud storage integration can also be added for long-term analysis and reporting.

On the hardware side, improvements such as higher sampling accuracy, better noise filtering, and support for higher-speed communication can enhance performance. The system can also be developed into a portable handheld device, making it suitable for industrial debugging, field testing, and educational purposes.

References

- [1] Espressif Systems, *ESP32 Technical Reference Manual*, Version 4.9; Espressif Systems, 2023.
- [2] M. Schwartz, *Mobile Wireless Communications*, Cambridge University Press, 2005, pp. 210–230.
- [3] OASIS Standard, *MQTT Version 3.1.1*, OASIS Open, Dec. 2014.
- [4] HiveMQ, *MQTT Essentials – A Lightweight IoT Protocol*
- [5] Arduino, *Arduino IDE Documentation*.
- [6] Texas Instruments, *Understanding SPI Communication*, Application Report, 2015.
- [7] NXP Semiconductors, *I2C-bus Specification and User Manual*, Rev. 6, Apr. 2014.
- [8] Microchip Technology Inc., *UART Communication Basics*, Application Note, 2018.
- [9] Wokwi, *Online ESP32 Simulator Documentation*